

Computer Network Word Duel Game: Project Explanation

This document provides a simplified yet comprehensive breakdown of the **Computer Network Word Game**, designed to help you explain its core concepts, architecture, and implementation details to your professor.

1. High-Level Overview

The project is a multiplayer, turn-based Word Guessing Game (similar to Hangman) that can be played over a local network. Players can join using a web browser or a custom Python desktop client (using Pygame).

Instead of using modern web frameworks like Flask, FastAPI, or Django, the server is built from scratch using raw **Python Sockets**. This demonstrates a deep understanding of core computer networking concepts.

2. Core Networking & System Concepts Demonstrated

When presenting to your professor, you should highlight these core concepts that your project implements from scratch:

A. Client-Server Architecture

The project strictly follows the client-server model:

- **Server:** Centralized authority that holds the game logic, player scores, and the secret word.
- **Clients:** The user interfaces (Web GUI or Pygame window) that send requests (join, guess, set word) to the server and display the game state to the player.

B. Socket Programming (Raw Sockets)

The server binds to an IP address (`0.0.0.0`) and port (`5555`) using Python's built-in `socket` library. It listens for incoming TCP connections. By doing this manually, the project demonstrates how data is moved across the transport layer before any application framework gets involved.

C. HTTP Protocol Implemented from Scratch

Usually, web frameworks handle HTTP requests. In this project, the server manually reads the raw bytes sent by the client, breaks down the headers, and parses the body.

- **Parsing:** It extracts the HTTP Method (GET, POST) and the Request Path (e.g., `/api/guess`).
- **Formatting:** It constructs the raw HTTP response headers (e.g., `HTTP/1.1 200 OK` , `Content-Type: application/json`) and appends the payload. This proves you understand how the Application Layer (HTTP) works under the hood.

D. Concurrency and Multithreading

To allow multiple players to interact with the server at the same time, the server uses **Multithreading**.

- Whenever a new connection is accepted by the socket, the server spawns a new background thread (`threading.Thread`) to handle that specific client's request.
- This prevents one slow client from blocking the whole game for everyone else.

E. Shared State & Thread Safety (Locks)

Because multiple threads are running at the same time, they might try to update the game score or game state simultaneously. This can cause a "Race Condition" (data corruption).

- To prevent this, the server uses a **Mutex Lock** (`threading.Lock()`).
- Whenever a thread wants to change the game state (e.g., process a guess or add a player), it must acquire the lock, make the change safely, and then release the lock.

F. RESTful JSON APIs

The communication between the clients and the server happens via JSON APIs. The server exposes endpoints such as:

- `POST /api/join` : To add a player.
- `GET /api/state` : To fetch the current status of the game, scores, and turns.
- `POST /api/guess` : For players to submit a letter or word guess.
- `POST /api/set-word` : For the "setter" to start the round.

G. Polling (Client-Side)

Because the server operates on standard HTTP (which is stateless), the server cannot "push" updates to the clients. Instead, the clients use a technique called **Polling**. Every second, the clients automatically send a `GET /api/state` request to the server to fetch the latest game updates.

3. How the Game Logic Works

1. **Roles:** Players join the lobby. The game automatically assigns one player as the **Setter** and the rest as **Guessers**.
 2. **Round Setup:** The Setter provides a secret word. The server masks it (e.g., "___") and tracks incorrect/correct guesses.
 3. **Turn-Based Sequence:** Guessers take turns making guesses. If a guesser gets a letter wrong, their turn ends and the next guesser plays.
 4. **Strikes:** If a guesser reaches the maximum number of mistakes (default 6 strikes = full hangman drawing), they are out for the round.
 5. **Winning:** A round is won when the word is fully revealed. The server awards a point. The first player to reach 3 points wins the match.
 6. **Rotation:** After a round ends, the role of "Setter" rotates down the player list so everyone gets a turn to pick the word.
-

4. Components Breakdown

- **server.py** : The heart of the network. It manages the continuous socket loop, parses raw HTTP strings, enforces game rules, handles locks, and serves static web files (like HTML/CSS/JS).
 - **client.py** : A desktop client built using Pygame. It uses Python's `urllib.request` to send API requests to the server, and draws the UI (including the Hangman drawing) locally based on the JSON response it gets.
 - **web/ Directory:** Contains the web-based client (`index.html` , `app.js`). This allows any device on the network (like a smartphone) to join the game through a standard web browser.
-

5. Summary Note for your Professor

"Professor, the goal of this project was not just to build a game, but to build the **underlying network infrastructure** required to host it. By relying on pure Python Sockets rather than a high-level framework, I successfully implemented multithreaded concurrent connections, manual HTTP framing, thread-safe memory management, and a RESTful API structure."